

FINAL PROJECT REPORT

Learning Management System(LMS)

An automated learning management system for educational institution

Built using MySQL, Python (Flask), and HTML/CSS

Course: Database System

Submitted by	Tuba (BSAI51S25R017) Ayesha Hassan (BSAI51S25R020) Emaan Fatima (BSAI51S25R042)
Submitted to	Sir Illyas
Department	Artificial Intelligence
University	University of Sargodha

Date: June 2026

Learning Management System

Table of Content

Abstract:

The Learning Management System (LMS) is a web-based database application developed to manage and automate educational operations. The system maintains records of user authentication, course enrollments, lecture materials, assignments, quizzes, and grading feedback. It provides three separate user roles — Administrator, Teacher, and Student — each with its own secure login and a dedicated interface tailored to their specific use cases.

The project applies core database concepts studied in this course: entity-relationship modelling, normalization up to Third Form, physical database design through indexing, and database security with concurrency control. The database is implemented in MySQL, the application logic is built using programming structures, and a clear user interface is provided. The result is a fully functional system that demonstrates how structured database systems and software requirements engineering (SRS) principles are applied to solve real-world educational challenges

1.Introduction:

A Database Management System (DBMS) is software that allows users to define, create, store, and manage data in an organized way, while enforcing rules that keep the data accurate, consistent, and secure. Instead of storing information in scattered files, a DBMS keeps it in related tables that can be queried and updated reliably.

Educational institutions handle large amounts of critical information daily: user registrations, course scheduling, study material distributions, assignment deadlines, and student grading sheets. Managing these workflows manually or via disconnected channels is slow, inefficient, and error-prone. The Learning Management System (LMS) solves this by storing all of this system data in a single, well-structured MySQL database and providing a seamless interface for academic collaboration.

This report explains the comprehensive Software Requirements Specification (SRS), architectural use cases, and the final database implementation of the system, showing clearly how each studied database and system design concept has been applied

2.Objectives and Scopes:

Objectives:

- To design a normalized relational database to manage academic and course data smoothly.
- To maintain organized records of user accounts, course materials, assignments, student attendance, and grades.
- To implement three secure and distinct user roles: Admin, Teacher, and Student

Learning Management System

- To apply database security, such as password protection, data privacy, and reliable backup controls.
- To provide a simple, user-friendly interface that loads quickly and allows easy access to learning resources.

Scopes:

The system is built to manage the core academic activities of an educational institution. It covers online student registration, course enrollment, lecture material management, and assignment submissions. It also allows teachers to manage grades, post announcements, and communicate with students directly through the platform. The system includes sample database records for courses, teachers, and student groups to demonstrate real-world academic management.

3.Tools and Technologies:

Technology	Purposes
MySQL (via XAMPP)	The relational database used to store all the tables like users, courses, and enrollments.
phpMyAdmin	A visual web interface used to easily create, modify, and manage the MySQL database schemas.
Python (Flask framework) / C++	The backend programming language used to handle the core application logic and connect the database. (Apni language ke mutabiq choose kar lein)
HTML and CSS	Used to design and build a clean, simple, and user-friendly frontend interface for browsers.
Cryptography / Security Functions	Used to securely hash and protect user passwords during registration and login.
Web browser	Used to run, test, and interact with the complete system locally.

4.System Modules:

The system is divided into three role-based modules. After logging in, each user sees only the options and dashboard relevant to their specific role.

Administrator Module:

The administrator manages the entire system. The admin can handle user accounts, manage student enrollments, schedule courses, and generate progress reports. Additionally, the admin has the authority to approve or reject new courses created by teachers before they go live.

Teacher Module:

A teacher logs in to a dedicated portal to create and manage their courses. They can upload learning materials like slides and PDFs, create assignments with deadlines, set up quizzes, and update the grade book. Teachers can also post important announcements and track student activities on the platform.

Student Module:

A student logs in to a personal dashboard where they can see their enrolled courses, download study materials, and read announcements. They can submit their assignments by uploading files, attempt quizzes, view their grades, and use the messaging feature to ask questions directly to their teachers.

5.Database Design:

Entities and Attribute:

The system is built around nine entities. Each entity becomes a table in our relational database:

- **Users** — Stores login credentials and account access details for all system participants (Admin, Teacher, Student).
- **Courses** — Academic classes or subjects created by teachers and approved by the admin.
- **Enrollments** — Tracks which student has joined which specific course.
- **Course Materials** — Lecture slides, study notes, videos, or PDFs shared by teachers.
- **Assignments** — Tasks and projects posted by teachers along with submission deadlines.
- **Submissions** — Homework files uploaded by students for evaluation.
- **Quizzes** — Online tests and assessments created by teachers to evaluate students.
- **Announcements** — Notice board posts created by teachers to share updates with the class.
- **Messages** — Direct text communication and queries between students and teachers.

Entity Relationship Diagram:

The following diagram shows the nine tables, their key attributes, and the foreign-key relationships that connect them. Primary keys are marked PK and foreign keys are marked FK.

Enhanced ER (EER) concepts are also used in this project. The **Users** table works like a supertype, and the role attribute (admin, teacher, student) works as a subtype discriminator to separate different users. **Enrollments** acts as an associative entity that connects students with their courses. Similarly, **Assignment Submissions** links students with their assignments to store homework data. Finally, **Course Schedule** and **Grade Book** tables help to link classes with their timings and marks smoothly.

Relationships:

- One teacher can create and manage many courses (one-to-many).
- One student can register for many courses through enrollments (one-to-many).
- One course can have multiple assignments, quizzes, and learning materials (one-to-many).
- One assignment can have multiple submissions from different students (one-to-many).
- A user account is linked to exactly one specific role profile, like an admin, teacher, or student (one-to-one).

6.Relational Schema:

Table	Primary Key	Foreign key	Purpose
Users	User-id		Stores basic login credentials and main roles for everyone.
Admins	Admin-id	User-id	Stores profile details for system administrators.
Teachers	Teacher-id	User-id	Stores teacher profiles, specializations,

Learning Management System

			and office hours.
Students	Student-id	User-id	Stores student profiles, majors, and registration years.
Courses	Course-id	Teacher-id	Stores course details like course codes and titles.
Enrollments	Enrollment-id	Student-id, Course-id	Tracks which student is registered in which course.
Course-schedule	Schedule-id	Course-id	Stores class days, start times, and end times.
Course-material	Material-id	Course-id	Stores lecture notes, files, and upload dates

announcement	Announcement-id	Course-id, teacher-id	Stores notice board posts and updates for classes.
assignment	Assignment-id	Course-id	Stores homework details, due dates, and total scores.
Assignment-submission	Submission-id	Assignment-id, student-id	Stores files uploaded by students and their marks.
Grade-book	Grade-id	student-id, Course-id	Stores final scores and

			performance remarks.
messages	Message-id	Sender-user-id, receipient-user-id	Stores personal chats and questions between users.

7.Normalization:

Normalization is the process of organizing tables to remove redundancy and avoid update problems. All tables in this database are normalized up to Third Normal Form (3NF).

First Normal Form(1NF):

A table is in 1NF if every column holds a single, atomic value and there are no repeating groups. For example, a student's email or password is stored as a single value. If a student enrolls in multiple courses, it is handled by separate rows in the `enrollments` table rather than adding multiple course columns in the student table.

Second Normal Form(2NF):

A table is in 2NF if it is in 1NF and every non-key column depends on the whole primary key. Because most tables use a single-column primary key (such as `course_id` or `user_id`), there are no partial dependencies. For associative tables like `enrollments` that use a composite structure, all details depend entirely on the combined keys.

Third Normal Form(3NF):

A table is in 3NF if it is in 2NF and no non-key column depends on another non-key column (no transitive dependency). For example, course details like the course title or description are not repeated inside the `course_materials` or `assignments` tables. Instead, these tables only store the `course_id`, and the main course details are kept safely once in the `courses` table.

8.Physical database Design:

Physical design decides how the data is actually stored and accessed efficiently. Appropriate data types were chosen for each column (for example, `ENUM` for user roles, `DATE` or `TIMESTAMP` for submission deadlines, and `VARCHAR` for usernames and email addresses).

Indexes were created on the columns that are searched most often, so that queries such as 'find all enrolled students in a course' or 'check submission status' run faster:


```
CREATE INDEX idx_user_role          ON users(role);  
  
CREATE INDEX idx_enroll_student     ON  
enrollments(student_id); CREATE INDEX idx_enroll_course  ON  
enrollments(course_id);  
  
CREATE INDEX idx_sub_assignment     ON assignment_submissions(assignment_id);
```

9.Database Security:

Several security measures protect the system and its data:

Password Hashing:

User passwords are never stored as plain text. They are securely hashed before being saved into the database. At login, the entered password is encrypted and compared with the stored hash.

This ensures that even if someone gets unauthorized access to the database, they cannot read the actual passwords.

Role-Based-Access-Control:

Each user has a specific role (admin, teacher, or student). The application checks this role before letting them open a webpage. For example, a student cannot open administrative setup pages, and a teacher cannot change backend system settings. This security is strictly enforced in the backend logic:

```
@app.route('/admin/dashboard')  
  
@role_required('admin') # only admins can open  
  
this def admin_dashboard():  
  
    ...
```

SQL Injection Prevention:

Every database query uses parameterized inputs (placeholders) instead of mixing raw strings with SQL commands. This completely stops malicious users from executing harmful database commands through input forms, effectively preventing SQL injection attacks:

```
cur.execute('SELECT * FROM users WHERE username = %s', (username,))
```

10.Session Management:

After a successful login, a secure data session keeps the user logged in as they browse different pages. Once they click log out, the active session is completely cleared so that

nobody else can use the back button to access their account or study logs.

Concurrency Control:

Concurrency control ensures that the database stays correct even when several actions happen at the same time. The key example in this system is a student submitting an assignment at the exact deadline while a teacher is updating marks or locking submissions.

When a student submits an assignment, multiple database actions must happen together: the submission record must be saved, and the course activity log must be updated. If only one of these actions happened, the data would become inconsistent. This is handled using a database transaction with row locking:

```
conn.autocommit(False)      # begin transaction
# check assignment status and lock the row to avoid conflicting submissions
SELECT assignment_id FROM assignments
WHERE assignment_id = %s
FOR UPDATE;
INSERT INTO assignment_submissions (assignment_id, student_id,
file_path) ... UPDATE enrollments SET status = 'submitted' WHERE
student_id = %s ... conn.commit()
# both updates succeed

together
```

ACID Properties:

- **Atomicity** — Either the submission record and status update both succeed together, or neither does (ROLLBACK on error).
- **Consistency** — The student's academic profile and submission history records always stay logically correct.
- **Isolation** — FOR UPDATE locks the resource so that two overlapping requests (like resubmitting or grading at the same fraction of a second) do not corrupt the database state.
- **Durability** — Once the submission is committed, the student's record is permanently saved and will not be lost even during a sudden system shutdown.

11. Testing:

The system was tested by performing each operation and confirming the result in the database and on screen.

Testcase	Expected Result	outcomes
----------	-----------------	----------

Login with correct password	User logged in, redirected to role dashboard	Pass
Login with correct password	Access denied, error message displayed	Pass
Admin adds a new teacher/student account	New user appears correctly in the database users list	Pass
Teacher uploads course material	PDF/File link shows up on the student's dashboard	Pass
Teacher creates an assignment	Assignment appears under the course with the due date	Pass
Student submits an assignment	File uploads successfully, status changes to 'submitted	Pass
Student tries to open admin setup page	Access blocked by role check, unauthorized error shown	Pass

12.Installation and Execution

- Start XAMPP and run Apache and MySQL services.
- Open <http://localhost/phpmyadmin> in your browser and import the `lms_database.sql` file to set up all tables.
- Open the project folder in your terminal and install the needed libraries with: `pip install -r requirements.txt`
- Run the main application file with the command: `python app.py`
- Open <http://127.0.0.1:5000> in any web browser and log in using the default credentials (admin / admin123).

13.Conclusion and future Enhancement

The Learning Management System (LMS) successfully shows how practical database design works in a real application. It correctly uses essential concepts like ER modeling, data normalization, indexing, database security, and concurrency control. The final system gives a working, role-based web portal that keeps academic records organized, consistent, and secure for admins, teachers, and students.

Future enhancements could include: adding automated email or SMS notifications for deadline reminders, integrating a live video stream feature for online classes, adding an analytics dashboard with interactive charts to track student progress, and deploying the application to a live cloud server so users can easily access it anywhere over the internet.

Learning Management System

References

- Hoffer, J. A., Venkataraman, R., and Topi, H. Modern Database Management. Pearson.
- MySQL Documentation — <https://dev.mysql.com/doc/>
- Flask Documentation — <https://flask.palletsprojects.com/>